ChatGPT

Technical research report on a JLPT and Jōyō kanji Japanese-learning app

Executive summary

A Japanese-learning app that is both **JLPT-aligned (N5–N1)** and **Jōyō kanji–complete**, while training **reading, writing, and speaking**, is best treated as **three coordinated learning tracks** sharing one underlying content system: a **JLPT track** (test-aligned language knowledge + reading + listening), a **Kanji track** (Jōyō-based literacy and writing), and a **Skills track** (integrated reading, listening, speaking, and handwriting drills). The key to making this feasible as a solo, personal project is to **enforce a narrow MVP**: (a) high-quality kanji + vocabulary SRS, (b) a reading mode with controllable furigana, and (c) "shadowing-lite" speaking practice with simple feedback (transcript + confidence + user self-rating), rather than "full pronunciation coaching." [1]

Two research constraints materially affect product design:

- **"JLPT-styled" cannot mean reusing real JLPT questions.** The official organisers explicitly prohibit unauthorised copying/reproduction and note third-party copyrighted material in listening and some sample items. Your app can emulate *formats* (timed sections, question types), but you should author original items and/or use properly licensed corpora. [2]
- **Jōyō kanji are an official "general-use" guideline set** intended for contemporary Japanese in law, public documents, newspapers, magazines, and broadcasting, and the list is 2,136 character types (in the main table). This makes it a strong backbone for literacy progression and furigana policies. [3]

Technically, a robust architecture for a solo developer is:

- **Client-first, offline-capable** mobile app (store content + review queue locally; sync progress to server).
- Backend that primarily does **identity, sync, analytics, and content delivery**, while computationally heavy language features (tokenisation/furigana, OCR, handwriting recognition, STT/TTS) can be **on-device first** with cloud fallbacks. [4]

For SRS, you can start with **SM-2-style scheduling** (well understood, low implementation risk) and later upgrade to **FSRS-style scheduling/optimisation** once you have enough user review logs. Spaced (distributed) practice and retrieval practice have strong evidence bases in learning science, which supports your app's emphasis on review + testing. [5]

Recommended next steps are at the end as a concise checklist.

Assumptions and reference standards

Assumptions

Because you specified no constraints, this report assumes:

- Target platforms include **iOS + Android**, with an optional web companion (reading + account management).
- Budget is "hobby/low spend," but you are willing to use paid APIs for speech if needed.
- You want **JLPT alignment** to mean: practice that resembles official section structure, timing, and skills (language knowledge + reading + listening), while acknowledging JLPT does not directly test speaking or writing production. [6]
- You want **Jōyō completeness** to mean: cover all 2,136 characters progressively with readings, vocabulary, and writing practice. [3]

What "JLPT-aligned" means in practice

The official JLPT structure and score reporting shape how a "JLPT-styled training" app should model content:

- Test sections and timing vary by level (e.g., N1/N2 combine language knowledge + reading, with listening separate; N3–N5 split vocabulary, grammar+reading, listening). [7]
- Score reporting uses "scoring sections" and ranges (e.g., N1–N3 sections scored 0–60 each; total 0–180; N4/N5 have a combined language knowledge+reading section scored 0–120 plus listening 0–60). [8]
- Officially, JLPT measures comprehensive competence through three elements: **Language Knowledge**, **Reading**, and **Listening**. [9]
- Scaled scores are based on a statistical method (Item Response Theory) and depend on answer patterns, not simple raw percentages, which implies your "mock JLPT score" should be marketed as *practice performance* rather than an equivalent score. [10]

What "Jōyō kanji" means for your app

The official Jōyō list (general-use kanji table) is described as guidance for kanji usage in modern Japanese across public life, and explicitly states the main table includes **2,136** character types. [3]

This supports a design where:

- Your Kanji track uses Jōyō as the literacy spine.
- Your reading mode's furigana policy can be "partial ruby by default" where Jōyō-known characters may hide furigana progressively, matching typical Japanese publishing practice (fully-annotated vs partially-annotated ruby is a known distinction in publishing accessibility discussions). [11]

Copyright reality check for JLPT content

JLPT test questions are copyrighted by the organisers, and unauthorised copying/reproduction is prohibited; sample questions additionally warn that some items and **all listening audio** include third-party copyrighted works and require separate permission. [12]

Implication: you should build **original** question banks and reading/listening materials, and/or use properly licensed corpora (see "Content sourcing and licensing").

Product vision and feature system

Prioritised product vision

A coherent product vision that is meaningfully "Japanese-specific" (beyond generic gamified apps) is:

- A **literacy-first** trainer: reading Japanese is the highest leverage skill, so the app emphasises *comprehensible reading with controlled furigana + vocabulary/kanji review*, while still providing speaking and handwriting practice as supporting skills.
- A **dual curriculum**: JLPT progression (N5→N1) plus Jōyō coverage (2,136 kanji), with a shared review engine. [13]
- A **format-faithful practice layer**: the app simulates official section timing and item styles without copying copyrighted materials. [14]

MVP scope for a solo developer

A realistic MVP that still feels "complete" should include:

- Accounts + cross-device sync (optional but valuable).
- SRS-based learning for vocabulary + kanji + grammar points. [15]
- A Japanese reading mode with furigana toggles (partial/none/full) built on ruby markup concepts. [16]
- Listening practice with TTS audio (for vocab/sentences) and a curated set of recorded audio for higher naturalness later (licensing-dependent). [17]
- Speaking practice as "shadowing-lite": play audio → record user → transcribe → highlight mismatches + allow user self-rating (confidence + self-report). The MVP should avoid promising phoneme-level pronunciation coaching unless you adopt a specialised API such as pronunciation assessment. [18]
- Handwriting practice: stroke order visualisation and a simple "draw → recognise" loop (or "draw → reveal correct" if recognition is deferred), using on-device digital ink recognition if available. [19]

Detailed feature list by track

JLPT track features

- Level paths N5–N1 with section-aware study plans (Language Knowledge, Reading, Listening). [20]
- Timed "section drills" that mirror official durations (e.g., N2: language knowledge+reading block + listening block). [7]
- Mock scoring shown as "practice section scores" rather than a claim of equivalence to official scaled scoring (since official scaling is IRT-based and not replicable without item calibration). [21]
- Grammar point library with examples, conjugation notes, and question templates (original).
- Vocabulary lists and sentence sets aligned to the JLPT level progression (sourced/licensed or self-authored; see licensing section).

Kanji track features

- Jōyō kanji progression (2,136) with metadata: readings, example words, stroke order, common compounds. [3]
- "Daily moji" (character of the day): show meaning gloss, readings, 3–6 example words, and a micro-writing drill.
- Stroke order animation and decomposition view (radical/components) using vector stroke datasets (e.g., KanjiVG). [22]
- Writing practice session types: trace mode, freehand mode, speed/accuracy mode.
- Recognition options: on-device digital ink recognition; optionally commercial handwriting SDKs for higher quality. [23]

Skills track features

- Reading: graded texts with furigana control, tap-to-lookup, and sentence mining into SRS. [24]
- Listening: sentence audio, dictation, minimal pairs (later), and timed comprehension questions.
- Speaking: shadowing workflow (prompt → record → transcript → compare) and "repeat after me" drills. [25]
- Writing: handwriting input for kana/kanji; optional camera-based OCR for "look up kanji from the world" (later). [26]

Exercise templates

The following templates are designed to be reusable across N5–N1 and across kanji/vocab/grammar content.

Language Knowledge templates (JLPT-like)
- Multiple choice meaning/usage (word or grammar point).
- Cloze deletion with distractors (particle/conjugation).
- Sentence reorder (build grammatical sentence).
- Transformation (plain → polite; active → passive; causative; etc.).
These mirror the "language knowledge" concept without copying official items. [20]

Reading templates
- Single paragraph gist question.
- Detail retrieval question.
- Reference question ("What does X refer to?").
- Matching headings to paragraphs.
Your reading mode should support ruby annotations for furigana display. [27]

Vocabulary templates
- JP→EN recall (or JP→definition in JP for advanced).
- Audio→kana/kanji typing.
- Collocation selection (choose the natural partner word).
- Minimal sentence completion ("Which word fits?").

Kanji templates
- Recognition: kanji → meaning + reading.
- Production: meaning/reading → write kanji (digital ink). [28]
- Component breakdown: pick the correct radical/component set.
- Stroke order: "which stroke comes next?" (derived from stroke data ordering). [29]

Speaking templates
- Shadowing-lite: listen → record → transcript compare (STT) → user self-assessment. [30]
- Prompted reading aloud (text shown; compare transcript).
- Pronunciation assessment variant (advanced): use a cloud feature; note some Japanese phoneme-level details may be limited depending on provider implementation. [31]

Content model, database schema, and SRS

Content model overview

A scalable model separates:

- **Canonical language objects** (kanji, vocab, grammar points, sentences, readings, audio).
- **Curriculum mapping** (JLPT level tags, Jōyō membership, lesson ordering, prerequisites).
- **User learning state** (SRS scheduling per item per skill, error history, mastery estimates).

This separation is important because "one item" can have multiple skills (recognise vs produce; read vs pronounce), and each skill benefits from its own scheduling state. This is consistent with the idea that spaced practice benefits depend on how the learner performs over time. [32]

Relational DB schema in table form

The following schema is designed for **PostgreSQL** (or another relational DB), but the entity boundaries also map to document databases if needed. (Implementation note: keep "content" mostly immutable/versioned; keep "user state" mutable.)

Table	Purpose	Key fields (examples)	Relationships / constraints
`users`	Account identity	`id (uuid)`, `email`, `created_at`, `locale`, `time_zone`	1→many with `user_item_state`, `study_events`
`content_versions`	Track versioning of content sets	`id`, `name`, `created_at`, `notes`	Content rows reference `content_version_id`
`jlpt_levels`	N5–N1 taxonomy	`id`, `code (N5..N1)`, `order`	Used by mapping tables; aligns to official level structure [33]
`kanji`	Canonical kanji record	`id`, `literal`, `joyo_flag`, `grade (optional)`, `stroke_count`	`joyo_flag` based on official Jōyō inclusion [3]
`kanji_readings`	On/Kun readings	`id`, `kanji_id`, `reading_kana`, `type (on/kun)`, `is_common`	many readings per kanji

Table	Purpose	Key fields (examples)	Relationships / constraints
`vocab`	Vocabulary entries	`id`, `lemma_kanji`, `lemma_kana`, `pos`, `gloss`, `frequency_rank (optional)`	link to `vocab_senses`, `vocab_kanji_map`
`vocab_senses`	Multi-sense meanings	`id`, `vocab_id`, `sense_index`, `gloss`, `usage_notes`	many senses per vocab
`vocab_kanji_map`	Which kanji appear in a word	`vocab_id`, `kanji_id`, `role`	many-to-many
`grammar_points`	Grammar items	`id`, `slug`, `title`, `explanation_md`, `level_hint`	maps to JLPT tags
`sentences`	Sentence bank	`id`, `text_ja`, `reading_kana (optional)`, `translation_en (optional)`, `license_code`	may come from licensed corpora [34]
`sentence_tokens`	Tokenisation result for furigana	`sentence_id`, `token_index`, `surface`, `lemma`, `reading`, `pos`	generated via tokenizer libs; supports ruby display [35]
`audio_assets`	Audio metadata	`id`, `asset_type (tts/human)`, `provider`, `voice`, `text_hash`, `storage_url`, `license_code`	used by listening/speaking drills [36]
`exercise_templates`	Reusable template definitions	`id`, `template_type`, `schema_json`, `render_hints`	referenced by `exercises`
`exercises`	Instantiated questions	`id`, `template_id`, `content_refs (json)`, `jlpt_level_id`, `difficulty`	no copyrighted JLPT content; original only [37]
`user_item_state`	Per-user SRS state per “item+skill”	`user_id`, `item_type`, `item_id`, `skill_type`, `due_at`, `stability`, `difficulty`, `interval_days`, `ease`	composite PK; drives reviews [38]

Table	Purpose	Key fields (examples)	Relationships / constraints
`study_events`	Event log	`id`, `user_id`, `timestamp`, `item_type`, `item_id`, `skill_type`, `result`, `latency_ms`, `device`	feeds analytics and SRS optimisation [39]
`user_settings`	Preferences	`user_id`, `furigana_mode`, `romaji_mode`, `srs_retention_target`, `privacy_flags`	supports furigana UX [40]

ER diagram

```
erDiagram
  users ||--o{ user_item_state : has
  users ||--o{ study_events : logs
  jlpt_levels ||--o{ exercises : tags

  kanji ||--o{ kanji_readings : has
  vocab ||--o{ vocab_senses : has
  vocab ||--o{ vocab_kanji_map : uses
  kanji ||--o{ vocab_kanji_map : appears_in

  sentences ||--o{ sentence_tokens : tokenised_into
  audio_assets }o--|| sentences : can_voice

  exercise_templates ||--o{ exercises : instantiates
```

(ER structure reflects the need to separate stable content from per-user scheduling state.) [15]

SRS algorithm options and recommended parameters

Evidence basis for SRS-style learning

Spaced (distributed) practice shows strong benefits across many studies, including a large meta-analysis; retrieval practice (testing) is also a high-utility learning technique in major reviews. [41]

Practical algorithm choices

Option	What it is	Pros	Cons	Good MVP choice?	Primary references
Leitner boxes	Bucketed intervals based on correctness	Easy to implement and explain	Coarse scheduling; weaker personalisation	Acceptable for very early MVP	(General spaced repetition framing) [39]
SM-2 style	Tracks easiness factor + intervals; classic algorithm	Well documented; widely implemented; simple state	Requires careful tuning of lapses/ learning steps	**Yes**	[42]
FSRS-style	Modern scheduler + optional optimiser for user memory	Better retention efficiency potential; learn user parameters	More complex; needs data to shine	Later phase	[43]

Recommended MVP parameter defaults (SM-2-inspired)

Base your first release on a conservative variant of SM-2:

- Maintain per item+skill: `ease`, `interval`, `repetitions`, `lapses`, `due_at`. [44]
- Use a short "learning" stage with multiple same-day steps (app-defined), then graduate into longer intervals; this mirrors why popular tools modify SM-2 for a learning stage rather than punishing early failures. [45]
- Expose a **retention target** setting (e.g., 85–90%) but implement it later if adopting FSRS; FSRS implementations typically include "optimiser" concepts that fit user memory patterns. [43]

Multi-skill scheduling strategy (important for your scope)

Treat "the same content" as multiple reviewable items by skill:

- Example: vocab item "食べる" can have separate states: *recognise meaning from JP*, *produce JP from meaning*, *listen→meaning*, *shadowing pronunciation*.
- This avoids the common failure mode where a learner can recognise but cannot produce/write. It also makes it feasible to add writing/speaking without breaking the core system.

UX patterns for reading, writing, and speaking

Furigana handling and reading mode

Ruby annotation is the standard web mechanism to show pronunciation guides (furigana) over East Asian text. HTML defines `<ruby>` and related elements, explicitly noting Japanese furigana as a primary use case. [16]

Recommended UX patterns:

- **Three-state furigana toggle**:
- *Full*: furigana on all kanji in texts (accessibility and beginner-friendly).
- *Partial*: furigana only on "unknown" kanji (personalised); aligns with common partially-annotated ruby practice. [46]
- *Off*: none (advanced).
- **Tap-to-reveal**: tap a word to temporarily show furigana + definition; this keeps the page readable.
- **Reading progress gating**: require "read to end" or "complete comprehension checks" before allowing sentence mining, to reduce random hoarding.

Implementation note: if you build a web reader, EPUB 3 explicitly supports ruby annotations in its XHTML content model, which makes it feasible to reuse the same rendering approach for in-app graded readers. [47]

Furigana generation and tokenisation UX

Two practical approaches:

- **On-device / in-app tokenisation + furigana generation**: use a Japanese morphological analyser and a furigana library. For example, a furigana conversion library such as Kuroshiro (MIT) is designed to convert Japanese sentences to kana/romaji with furigana modes, commonly paired with a tokenizer like Kuromoji.js (Apache 2.0). [48]
- **Precomputed tokenisation** for curated texts: ship `sentence_tokens` as part of your content pack to guarantee stable furigana and avoid runtime variability.

Handwriting input and writing feedback flow

A writing UX that is achievable in MVP:

1. Show kanji with stroke order animation (vector strokes).
2. User writes on a grid canvas ("genkō yōshi" style).
3. The system recognises the character and gives immediate "correct / close / incorrect," then shows the model answer and encourages a repeat.

On-device recognition options:

- **Digital ink recognition**: ML Kit's digital ink recognition recognises handwritten text from stroke input (not camera images) and supports hundreds of languages and scripts; it's positioned for handwriting input on touch surfaces rather than OCR. [49]
- **Commercial handwriting SDKs** (higher quality, licensing cost): MyScript provides an SDK with per-device licensing model. [50]

For stroke order visuals:

- KanjiVG provides SVG stroke order diagrams and documents that the strokes are ordered according to stroke order references; it also documents the internal SVG group structure. Licensing is CC BY-SA 3.0. [22]

Speaking and shadowing flow

For the MVP, focus on **shadowing-lite**:

- Play audio (TTS or recorded).
- Record the learner.
- Run speech-to-text to produce a transcript, and highlight differences from the reference.
- Ask for self-rating ("Easy / OK / Hard") to augment noisy STT confidence.

Web option:

- The Web Speech API provides speech recognition and speech synthesis components, but speech recognition has limited cross-browser availability and often relies on platform services. [51]

Native option:

- iOS requires explicit permission to use speech recognition via the Speech framework (requesting authorisation). [52]
- Android speech recognition is provided via `SpeechRecognizer` and requires `RECORD_AUDIO` permission. [53]

Advanced option (later phase):

- Pronunciation assessment APIs can provide accuracy/fluency style feedback; Azure documents pronunciation assessment capabilities, including Japanese availability announcements, but implementation details and granularity can vary (some reported limitations exist for Japanese phoneme output). [54]

Technology stacks and third-party integrations

Recommended stacks for mobile and web

Below are three realistic stack options, ranked by "solo developer leverage."

Stack	Frontend	Backend	DB	Hosting	Pros	Cons	Cost/ complexity estimate
Flutter + Postgres platform	Flutter mobile (and optional Flutter web) [55]	Serverless functions / thin API	Postgres	Managed Postgres platform + CDN	Single codebase, strong UI control, multi-platform	Fewer "Japanese NLP" libs in Dart; may rely on services for tokenisation	Low-to-medium monthly cost; medium complexity

Stack	Frontend	Backend	DB	Hosting	Pros	Cons	Cost/ complexity estimate
React Native (Expo) + BaaS	React Native + Expo CI/ CD workflows [56]	BaaS functions	Firestore or Postgres	Firebase Hosting / BaaS hosting	Fast iteration and deployment automation; web parity via JS ecosystem	Native handwriting and offline can require extra work; vendor lock-in risk (Firestore)	Low-to-medium cost; low-to-medium complexity
Native iOS + Android	Swift + Kotlin	Custom API	Postgres	Any cloud	Best access to handwriting/ speech UX	Highest dev time for solo	Higher complexity/ time; variable cost

Notes on platform/build automation:

- Expo provides build and submission workflow automation (EAS workflows), lowering operational friction for solo developers. [57]
- Flutter positions itself as natively compiled and multi-platform from a single codebase, which is attractive for a personal project. [58]

Infrastructure and hosting primitives

A pragmatic architecture is:

- **Content delivery**: static JSON/SQLite content packs via CDN; version via `content_versions`.
- **User/state API**: auth + sync + events + leaderboards.
- **Media**: audio in object storage with signed URLs.

If you choose Firebase, note:

- Firebase provides a pricing page including no-cost quotas for multiple components. [59]
- Firestore's free tier is explicitly quantified (storage + read/write/delete quotas) and paid pricing is per operation units. [60]

If you choose Supabase, its pricing page indicates paid plans begin with monthly compute credits (exact free-tier limits can change, so treat public blog summaries cautiously). [61]

Speech, handwriting, and OCR provider comparison

Speech-to-text and text-to-speech providers

Provider	Capabilities	Example endpoint / API	Pricing model (indicative)	Integration r
Cloud Speech-to-Text (Google)	STT via REST; synchronous recognise supported	`POST https://speech.googleapis.com/v1/speech:recognize` [62]	Per minute of audio; pricing page shows free minutes tier and paid rate tiers [63]	Requires clo auth; good f server-side processing
Cloud Text-to-Speech (Google)	TTS via REST	Base endpoint `https://texttospeech.googleapis.com` [64]	Per character; pricing page provides per-character rates and free tier [65]	Good for generating consistent au for vocab/ sentences
AWS Transcribe	Batch transcription via jobs	`StartTranscriptionJob` requires media in S3 [66]	Per minute tiers [67]	Operational overhead: S3 upload + job polling
AWS Polly	TTS, including voice tiers	`SynthesizeSpeech` with input limits [68]	Per 1M characters; multiple voice tiers and free tier info [69]	Solid for TTS watch reque quotas/limits
Azure Speech	STT + TTS + pronunciation assessment	STT short-audio endpoint pattern: `https://<REGION>.stt.speech.microsoft.com/...` [70]	Billing is per second (STT) and per character (TTS); official docs describe billing character counting and note kanji count impacts [71]	Pronunciatio assessment available, bu short-audio has limitatio (duration cap [72]
iOS native	AVSpeechSynthesizer (TTS) + Speech framework STT	Platform APIs [73]	OS-provided	Strong priva story; STT re permission

Provider	Capabilities	Example endpoint / API	Pricing model (indicative)	Integration r
Browser	Web Speech API	`SpeechRecognition`, `SpeechSynthesis` [75]	Browser/ platform dependent	SpeechReco not broadly available cro browser [76]

Important TTS billing nuance if using Azure: a documented pricing note states billing is based on characters and that each Chinese character is counted as two characters, including kanji used in Japanese. This directly affects cost estimates for kanji-heavy synthesis. [77]

Handwriting recognition and OCR

Need	Option	Strengths	Caveats	References
Handwriting (digital ink)	ML Kit Digital Ink Recognition	Designed for stroke input; supports many scripts; on-device feel	Recognition quality varies; you must collect stroke points (not camera images)	[78]
Handwriting (premium)	MyScript iink SDK	High-quality handwriting recognition; device licensing	Licensing cost + integration complexity	[50]
OCR (camera text)	ML Kit Text Recognition v2	Explicitly supports Japanese script in v2	Camera OCR is a separate UX from handwriting	[79]
OCR (open source)	Tesseract	Apache 2.0 licensed OCR engine	Quality varies; Japanese OCR requires tuning	[80]
OCR (iOS)	Vision text recognition	Native Vision framework request type exists	Supported languages may vary by OS; query supported languages at runtime	[81]

Libraries and datasets for Japanese language features

Use case	Recommended resource	Licence / constraints	Notes
Dictionary entries (vocab, readings)	EDRDG dictionaries such as JMdict	CC BY-SA 4.0 (share-alike + attribution) [82]	Ensure you can comply with share-alike in redistributed data
Kanji readings and metadata	KANJIDIC files	CC BY-SA 4.0 [83]	Useful for readings and kanji info
Stroke order vectors	KanjiVG SVG files	CC BY-SA 3.0 [84]	Strokes are ordered for stroke order [29]

Use case	Recommended resource	Licence / constraints	Notes
Sentence corpora	Tatoeba corpus	Default CC BY 2.0 FR; attribution required [85]	Audio licences vary; track per-asset licence
JP–EN classic sentence base	Tanaka Corpus	Public domain origins; distributed as CC BY via Tatoeba context [86]	Attribution required in CC BY form
Tokenisation	Kuromoji.js	Apache 2.0 [87]	Good for JS stacks
Tokenisation	SudachiPy	Apache 2.0 [88]	Strong modern tokenizer; dictionary editions available [89]
Tokenisation	MeCab	Tri-licensed (BSD/GPL/LGPL) in common distributions [90]	Powerful; packaging can be OS-specific
Furigana conversion	Kuroshiro	MIT (common distributions) [91]	Works well with Kuromoji-based tokenisation
Local STT	Whisper + whisper.cpp	MIT licence [92]	Enables offline transcription at cost of on-device compute

Content sourcing, licensing, security, operations, and roadmap

Content sources and licensing strategies

Because JLPT questions are closed and copyrighted, your safest strategy is to:

- Use **open datasets** for foundational lexicon and sentences; author your own grammar explanations and questions; and strictly avoid scraping copyrighted question banks. [93]
- Maintain a `license_code` and `attribution_text` for every imported content object (sentence, audio, dictionary entry set) so you can generate an in-app attribution page.

A practical content-source comparison:

Content type	Source strategy	Pros	Risks / obligations	References
JLPT-style questions	Self-authored templates + generated distractors	Fully ownable content	Must avoid copying; quality requires iteration	[37]
Dictionary data	Import JMdict/KANJIDIC	Rich coverage	CC BY-SA share-alike obligations	[94]
Stroke order	KanjiVG	High-quality vectors	CC BY-SA 3.0 share-alike	[84]

Content type	Source strategy	Pros	Risks / obligations	References
Example sentences	Tatoeba/Tanaka	Large corpus	Attribution required; audio licences vary	[34]
Listening audio	Start with TTS; later license recordings	Fast scalability	TTS costs; voice licensing policies; pronunciation models	[95]

Licensing pitfall: share-alike mixing

If you combine CC BY-SA datasets (e.g., JMdict CC BY-SA 4.0 and KanjiVG CC BY-SA 3.0), you must respect each licence's share-alike and attribution requirements. Plan for:

- Clear attribution UI (Settings → Credits).
- Re-releasing your *derived dataset* under a compatible share-alike licence where required. [96]

Deployment, scaling, and analytics plan

A scalable plan for a personal project:

- **Phase 1**: single-region deployment; Postgres + object storage; CDN for static content packs.
- **Phase 2**: add background jobs for content processing (tokenisation, TTS generation, audio normalisation).
- **Phase 3**: add caching for reading endpoints; rate-limit and quota expensive speech endpoints.

Analytics that matter for learning outcomes:

- SRS health: daily review count, overdue percentage, retention estimate by skill.
- Content quality signals: "reported bad card," high lapse rate, confusing distractor detection.
- Funnel: activation (first review), D7 retention.

If using Firestore/Firebase, cost drivers are reads/writes/egress; the pricing pages and quota pages provide explicit free tiers and unit pricing. [97]

Security, privacy, and data retention considerations

Your app potentially processes sensitive inputs: voice recordings, handwriting traces, and learning behaviour.

Security baselines:

- Use established mobile security standards such as the OWASP Mobile Application Security Verification Standard as a baseline for requirements and testing. [98]
- Follow secure session management practices (token rotation, secure storage, re-auth on risk events). [99]

Privacy constraints and permissions:

- iOS speech recognition requires user authorisation; Android speech recognition requires microphone permission. Treat these as "high friction" permissions: request only when entering speaking practice. [100]

- If you ever use voice “to uniquely identify someone,” regulators treat biometric data as more sensitive (“special category”) under UK GDPR concepts; therefore, avoid identity use-cases and do not market voice biometrics. [101]

Data retention principle:

- UK regulator guidance emphasises keeping personal data only as long as needed; there are no universal time limits, so you must define purpose-driven retention policies. [102]

Practical retention defaults (reasonable for MVP):

- Audio recordings: **off by default**, transient processing; delete server-side immediately after transcription unless user opts in to save for progress comparison.
- Handwriting traces: store only derived correctness metrics by default; optionally store traces locally for replay.
- Study events: keep aggregated analytics long-term; raw event logs shorter (e.g., 90–180 days), unless user opts in.

Roadmap with phases and solo-developer time estimates

Time estimates assume part-time solo development and initial content assembly.

```
gantt
  title Roadmap for a JLPT + Jōyō Japanese-learning app
  dateFormat  YYYY-MM-DD
  axisFormat  %b %Y

  section Foundation
  Product spec + schema + licensing plan     :a1, 2026-03-04, 21d
  Core content import (kanji + vocab)        :a2, after a1, 28d

  section MVP Build
  Mobile app skeleton + offline store        :b1, after a2, 28d
  SRS engine + review UI                     :b2, after b1, 28d
  Reading mode + furigana toggle             :b3, after b2, 21d

  section Skills Add-ons
  Handwriting practice (digital ink)         :c1, after b3, 21d
  Speaking shadowing-lite (STT + compare)    :c2, after c1, 21d

  section Quality and Release
  Analytics + QA + security baseline          :d1, after c2, 21d
  Closed beta + iteration                     :d2, after d1, 28d
```

The ordering focuses on getting a “compounding core” working first: SRS + reading. This matches evidence that distributed practice and retrieval practice are high-utility techniques, and it avoids premature complexity (full pronunciation coaching, large media pipelines). [103]

Recommended next steps checklist

- Define a written MVP contract: **SRS + reading mode + basic speaking/writing**, and explicitly defer advanced features (pitch accent coaching, JLPT "score equivalency," large-scale audio licensing). [104]
- Choose one stack: if unsure, pick **single-codebase mobile** and a **thin backend** (sync + auth + content delivery). [105]
- Finalise your licensing plan: decide whether you can comply with **CC BY-SA** obligations for dictionary/kanji datasets; implement a `license_code` + attribution system from day one. [106]
- Implement the data model and SRS state machine (SM-2 style) and ship a local-only prototype first. [42]
- Build the reading renderer with ruby support and a three-state furigana toggle. [16]
- Add handwriting using digital ink recognition (or defer recognition and start with guided tracing + reveal). [107]
- Add speaking as shadowing-lite (STT transcript + compare), and only later evaluate pronunciation assessment APIs. [108]
- Establish privacy/security baselines (OWASP MASVS-inspired checklist; minimal retention; permission gating for mic). [109]

---

[1] [6] [7] [8] [13] [14] [33] [104] https://www.jlpt.jp/reference/pdf/guidebook_s_e.pdf
https://www.jlpt.jp/reference/pdf/guidebook_s_e.pdf

[2] https://www.jlpt.jp/e/policy.html
https://www.jlpt.jp/e/policy.html

[3] [11] https://www.bunka.go.jp/kokugo_nihongo/sisaku/joho/joho/kijun/naikaku/pdf/joyokanjihyo_20101130.pdf
https://www.bunka.go.jp/kokugo_nihongo/sisaku/joho/joho/kijun/naikaku/pdf/joyokanjihyo_20101130.pdf

[4] [19] [23] [28] [49] [78] [107] https://developers.google.com/ml-kit/vision/digital-ink-recognition
https://developers.google.com/ml-kit/vision/digital-ink-recognition

[5] [38] [42] [44] SuperMemo 2: Algorithm
https://super-memory.com/english/ol/sm2.htm?utm_source=chatgpt.com

[9] [20] JLPT Japanese-Language Proficiency Test
https://www.jlpt.jp/sp/e/about/points.html?utm_source=chatgpt.com

[10] [21] https://www.jlpt.jp/e/about/pdf/scaledscore_e.pdf
https://www.jlpt.jp/e/about/pdf/scaledscore_e.pdf

[12] https://www.jlpt.jp/e/samples/sampleindex.html
https://www.jlpt.jp/e/samples/sampleindex.html

[15] [32] [39] [41] [103] https://pubmed.ncbi.nlm.nih.gov/16719566/
https://pubmed.ncbi.nlm.nih.gov/16719566/

[16] [24] [27] [35] [40] https://html.spec.whatwg.org/multipage/text-level-semantics.html
https://html.spec.whatwg.org/multipage/text-level-semantics.html

[17] [36] [64] https://docs.cloud.google.com/text-to-speech/docs/reference/rest
https://docs.cloud.google.com/text-to-speech/docs/reference/rest

[18] [31] [54] https://learn.microsoft.com/en-us/azure/ai-services/speech-service/how-to-pronunciation-assessment
https://learn.microsoft.com/en-us/azure/ai-services/speech-service/how-to-pronunciation-assessment

22 84 The Kanji Vector Graphics (KanjiVG) project - KanjiVG
https://kanjivg.tagaini.net/?utm_source=chatgpt.com

25 30 51 75 108 https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

26 79 https://developers.google.com/ml-kit/vision/text-recognition/v2/languages
https://developers.google.com/ml-kit/vision/text-recognition/v2/languages

29 SVG Format
https://kanjivg.tagaini.net/svg-format.html?utm_source=chatgpt.com

34 85 https://en.wiki.tatoeba.org/articles/show/using-the-tatoeba-corpus
https://en.wiki.tatoeba.org/articles/show/using-the-tatoeba-corpus

37 93 https://www.jlpt.jp/sp/e/faq/
https://www.jlpt.jp/sp/e/faq/

43 open-spaced-repetition/fsrs4anki: A modern Anki custom ...
https://github.com/open-spaced-repetition/fsrs4anki?utm_source=chatgpt.com

45 What spaced repetition algorithm does Anki use?
https://faqs.ankiweb.net/what-spaced-repetition-algorithm?utm_source=chatgpt.com

46 https://github.com/w3c/a11y-discov-vocab/issues/68
https://github.com/w3c/a11y-discov-vocab/issues/68

47 https://www.w3.org/TR/epub-overview-33/
https://www.w3.org/TR/epub-overview-33/

48 https://kuroshiro.org/
https://kuroshiro.org/

50 Pricing | MyScript Developer
https://developer.myscript.com/pricing?utm_source=chatgpt.com

52 74 100 https://developer.apple.com/documentation/speech/asking-permission-to-use-speech-recognition
https://developer.apple.com/documentation/speech/asking-permission-to-use-speech-recognition

53 https://www.dre.vanderbilt.edu/~schmidt/android/android-4.0/out/target/common/docs/doc-comment-check/reference/android/speech/SpeechRecognizer.html
https://www.dre.vanderbilt.edu/~schmidt/android/android-4.0/out/target/common/docs/doc-comment-check/reference/android/speech/SpeechRecognizer.html

55 58 105 https://flutter.dev/
https://flutter.dev/

56 https://reactnative.dev/
https://reactnative.dev/

57 https://docs.expo.dev/eas/workflows/get-started/
https://docs.expo.dev/eas/workflows/get-started/

59 https://firebase.google.com/pricing
https://firebase.google.com/pricing

60 97 https://cloud.google.com/firestore/pricing
https://cloud.google.com/firestore/pricing

61 https://supabase.com/pricing
https://supabase.com/pricing

62 https://docs.cloud.google.com/speech-to-text/docs/reference/rest/v1/speech/recognize
https://docs.cloud.google.com/speech-to-text/docs/reference/rest/v1/speech/recognize

63 https://cloud.google.com/speech-to-text/pricing
https://cloud.google.com/speech-to-text/pricing

65 95 https://cloud.google.com/text-to-speech/pricing
https://cloud.google.com/text-to-speech/pricing

66 https://docs.aws.amazon.com/transcribe/latest/APIReference/API_StartTranscriptionJob.html
https://docs.aws.amazon.com/transcribe/latest/APIReference/API_StartTranscriptionJob.html

67 https://aws.amazon.com/transcribe/pricing/
https://aws.amazon.com/transcribe/pricing/

68 https://docs.aws.amazon.com/polly/latest/dg/API_SynthesizeSpeech.html
https://docs.aws.amazon.com/polly/latest/dg/API_SynthesizeSpeech.html

69 https://aws.amazon.com/polly/pricing/
https://aws.amazon.com/polly/pricing/

70 72 https://learn.microsoft.com/en-us/azure/ai-services/speech-service/rest-speech-to-text-short
https://learn.microsoft.com/en-us/azure/ai-services/speech-service/rest-speech-to-text-short

71 77 https://docs.microsoft.com/en-us/azure/cognitive-services/speech-service/text-to-speech
https://docs.microsoft.com/en-us/azure/cognitive-services/speech-service/text-to-speech

73 https://developer.apple.com/documentation/avfaudio/avspeechsynthesizer
https://developer.apple.com/documentation/avfaudio/avspeechsynthesizer

76 https://developer.mozilla.org/en-US/docs/Web/API/SpeechRecognition
https://developer.mozilla.org/en-US/docs/Web/API/SpeechRecognition

80 Introduction | tessdoc - Tesseract documentation
https://tesseract-ocr.github.io/tessdoc/Installation.html?utm_source=chatgpt.com

81 https://developer.apple.com/documentation/vision/recognizing-text-in-images
https://developer.apple.com/documentation/vision/recognizing-text-in-images

82 94 96 106 https://www.edrdg.org/edrdg/licence.html
https://www.edrdg.org/edrdg/licence.html

83 https://www.edrdg.org/wiki/KANJIDIC_Project.html
https://www.edrdg.org/wiki/KANJIDIC_Project.html

86 https://www.edrdg.org/wiki/Tanaka_Corpus.html
https://www.edrdg.org/wiki/Tanaka_Corpus.html

87 https://github.com/takuyaa/kuromoji.js/blob/master/LICENSE-2.0.txt
https://github.com/takuyaa/kuromoji.js/blob/master/LICENSE-2.0.txt

88 https://github.com/WorksApplications/SudachiPy/blob/develop/LICENSE
https://github.com/WorksApplications/SudachiPy/blob/develop/LICENSE

89 https://pypi.org/project/SudachiPy/
https://pypi.org/project/SudachiPy/

90 https://sourceforge.net/projects/mecab/
https://sourceforge.net/projects/mecab/

91 https://github.com/Shuruni/kuroshiro-browser
https://github.com/Shuruni/kuroshiro-browser

92 openai/whisper: Robust Speech Recognition
https://github.com/openai/whisper?utm_source=chatgpt.com

98 109 https://mas.owasp.org/
https://mas.owasp.org/

99 https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

101 https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/lawful-basis/biometric-data-guidance-biometric-recognition/key-data-protection-concepts/
https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/lawful-basis/biometric-data-guidance-biometric-recognition/key-data-protection-concepts/

102 https://ico.org.uk/for-organisations/advice-for-small-organisations/information-security/data-storage-advice/
https://ico.org.uk/for-organisations/advice-for-small-organisations/information-security/data-storage-advice/